

GSD — Power-User Reference

Printable workflow loops, gates, and command cheat sheet. Each diagram fits one A4 page.

Mental model in 60 seconds

GSD is a thin orchestrator over Claude Code (or OpenCode / Codex / Gemini / Kilo / Copilot / Cursor / Windsurf / ...). It only does five things:

- 1. **Initialise** — `/gsd-new-project` (greenfield) or `/gsd-map-codebase + /gsd-new-project` (brownfield), or `/gsd-ingest-docs` (existing planning docs). Produces `.planning/` with `PROJECT.md` + `REQUIREMENTS.md` + `ROADMAP.md` + `STATE.md`.
- 2. **For each phase** — `discuss` → (optional design contracts) → `plan` → `execute` → (optional review) → `verify` → `ship`. Each command writes one artifact the next consumes (`CONTEXT.md` → `RESEARCH.md` → `PLAN.md` → `SUMMARY.md` → `VERIFICATION.md` → `UAT.md` → `PR`).
- 3. **Milestone close** — `/gsd-audit-milestone` → `/gsd-complete-milestone` → `/gsd-new-milestone` (or stop).
- 4. **Side-quests** — debug, retroactive audits, cross-AI peer review, undo, threads, backlog, quick fixes. These run *off* the main loop.
- 5. **Aggregators** — `/gsd-autonomous`, `/gsd-manager`, `/gsd-progress --next` batch the main loop so you don't have to.

The orchestrator never holds the heavy context — subagents do, each with fresh ~200 k tokens. Artefacts in `.planning/` are the persistent memory.

Legend: optional gate remediation observability capture standalone

Diagram 1 — Full project lifecycle

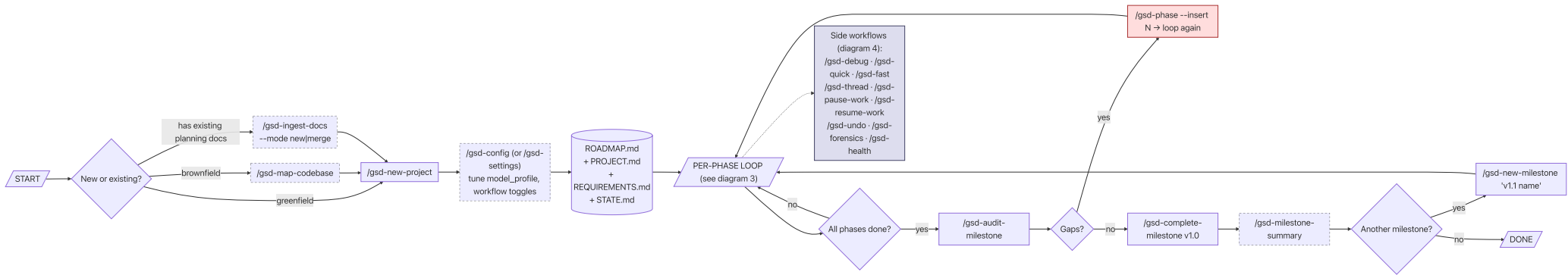


Diagram 2a — Main phase loop · plan side (spec → discuss → plan → execute)

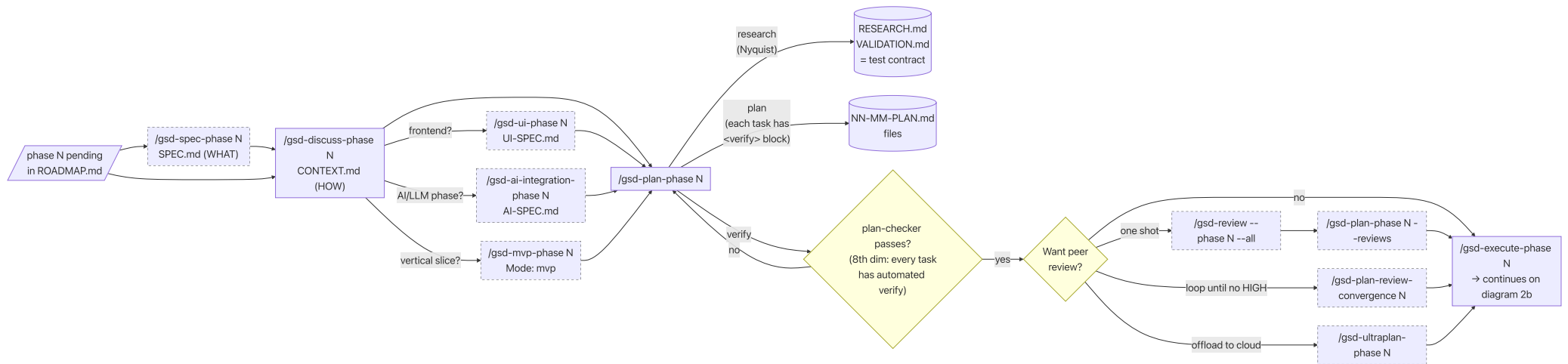
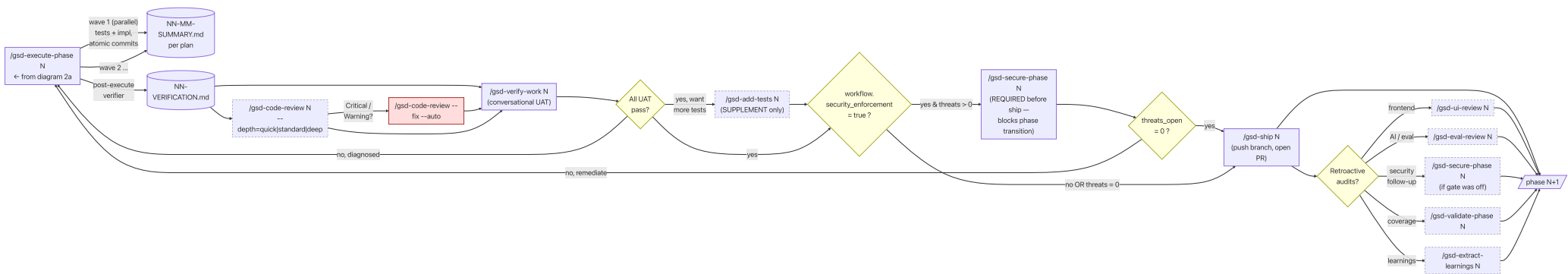


Diagram 2b — Main phase loop · verify & ship side (execute → UAT → security → ship)



Phase-loop rules & gates

Rules of thumb:

- `spec-phase` is the optional ambiguity-scoring gate; skip when the phase description is already crisp.
- `discuss-phase` is **the** must-do step — it locks "how" before any code.
- Pick **one** of `ui-phase`, `ai-integration-phase`, `mvp-phase` based on what the phase actually delivers. They are not mutually exclusive but rarely all needed.
- Peer-review options are **pre-execute**: `review` (one pass) → `plan-phase --reviews` (apply feedback) **or** `plan-review-convergence` (loop until no HIGH concerns).
- Some "retroactive" audits are actually **pre-ship gates** when their config toggle is on (the default). `verify-work` won't auto-transition the phase to complete until they pass:

| Audit | Pre-ship gate when... | Default | Where the gate fires | |---|---|---|---| | `/gsd-secure-phase N` | `workflow.security_enforcement: true` | **on** | end of `verify-work` | | `/gsd-validate-phase N` | `workflow.nyquist_validation: true` (via `plan-checker`'s 8th dim) | **on** | end of `plan-phase` (refuses plans without `verify` cmds) | | `/gsd-ui-review N` | `workflow.ui_safety_gate: true` (prompt, non-blocking) | **on** | end of `plan-phase` (frontend phases) | | `/gsd-eval-review N` | n/a — always retroactive | — | post-ship only | | `/gsd-audit-uat` | n/a — always retroactive | — | cross-phase, any time |

When the toggle is **off**, the same command becomes a purely optional retroactive audit you run after `ship`. When the toggle is **on** and you haven't run the command, the phase is **stuck in "verified but not complete"** state until you do.

- Remediation pattern is the same in both cases: the audit produces a remediation plan you feed back into `/gsd-execute-phase N --gaps-only`, then re-run the audit.

When tests get written (three layers)

Tests are **not** an after-thought of `verify-work`. There are three complementary layers, and most projects only use the first two:

When	Mechanism	What it does	Artifact
plan-phase	Nyquist validation (default, <code>workflow.nyquist_validation: true</code>)	Researcher maps every requirement → specific test command; identifies scaffolding that must exist before code (Wave 0 tasks). Plan-checker's 8th verification dimension blocks any plan whose tasks lack an automated <code><verify></code> command.	<code>{NN}-VALIDATION.md</code> — the phase's test contract
execute-phase	Per-task <code><verify></code> block in each PLAN file	Each executor writes tests alongside the implementation in the same atomic commit. With <code>--tdd</code> , planner orders tasks as RED → GREEN → REFACTOR so the failing test is committed before the fix.	tests + impl in <code>{NN}-MM-SUMMARY.md</code>
post-ship (optional)	<code>/gsd-add-tests N</code>	Supplement only — generates extra unit / E2E tests against <code>SUMMARY.md</code> / <code>CONTEXT.md</code> / <code>VERIFICATION.md</code> when you want more than the plans produced (fuzz, edge cases, regression seeds, UAT-discovered gaps). Classifies into TDD/E2E/Skip; you approve before commit.	<code>test(phase-{N}): ...</code> commits

Retroactive equivalents (for phases shipped before Nyquist was on, or brownfield code with thin coverage):

- `/gsd-validate-phase N` — audits which requirements lack automated checks and **generates the missing tests** (auditor never touches impl code — if a test reveals a bug, it's escalated to you).
- `/gsd-audit-uat + /gsd-audit-fix` — cross-phase scan for outstanding UAT items, then autonomous fix pipeline.

TDD-specific flags:

- `/gsd-plan-phase --tdd` — planner orders each plan's tasks RED → GREEN → REFACTOR.
- `/gsd-execute-phase --tdd` — executor enforces the RED step actually fails before the GREEN commit is allowed.

- `/gsd-mvp-phase` — orthogonal: organises plans as **vertical slices** (UI→API→DB) per feature instead of horizontal layers. Combine with `--tdd` for slice-level TDD.
- Disable Nyquist** (rapid prototyping only): set `workflow.nyquist_validation: false` via `/gsd-settings`. Plan-checker stops enforcing the verify-command dimension; you'll have to manage test coverage manually.

nyquist_compliant frontmatter — when does it flip to true?

`{NN}-VALIDATION.md` is written by `/gsd-plan-phase` from a template that **defaults the flag to false**:

```
---
phase: N
status: draft
nyquist_compliant: false  ← default; never auto-flipped
wave_0_complete: false
---
```

Nothing in the standard loop (`plan-phase`, `execute-phase`, `verify-work`, `ship`) ever flips it. Only `/gsd-validate-phase N` does (via the `gsd-nyquist-auditor` agent) once it confirms every requirement has automated verification and the sign-off checklist passes:

- **COMPLIANT** — `nyquist_compliant: true` AND every per-task row  green.
- **PARTIAL** — file exists, flag still `false`, some rows  /  or escalated to manual-only.
- **MISSING** — no `VALIDATION.md` (only happens with `workflow.nyquist_validation: false`).

`/gsd-audit-milestone` reads each phase's `VALIDATION.md` and flags `nyquist_compliant: false` phases as **PARTIAL**, recommending you run `/gsd-validate-phase N` for each before `/gsd-complete-milestone`. You can do this per-phase or batch it once at milestone close — either works.

Adversarial review — when to run which one

GSD ships two adversarial reviews, both explicitly described as *adversarial* in their agent prompts. The `gstack /codex challenge` skill is a useful third-opinion add-on if installed.

Tool	Target	Stance	Position	Output
<code>/gsd-review --phase N --all</code>	Plans (pre-execute)	External AI CLIs (Gemini, Claude, Codex, OpenCode, Qwen, Cursor) peer-review the PLAN files	between <code>plan-phase</code> and <code>execute-phase</code>	<code>{NN}-REVIEWS.md</code> (feed back via <code>plan-phase --reviews</code>)
<code>/gsd-plan-review-convergence N --all [--max-cycles M]</code>	Plans (pre-execute)	Same as above, looped until no HIGH concerns	same	replans + final <code>REVIEWS.md</code>
<code>/gsd-code-review N --depth=deep</code>	Shipped code	<code>gsd-code-reviewer</code> agent with explicit "FORCE stance: assume every implementation contains defects". BLOCKER / WARNING classification only — no "INFO" downgrades.	between <code>execute-phase</code> and <code>verify-work</code>	<code>{NN}-REVIEW.md</code>
<code>/gsd-code-review N --fix --auto</code>	Shipped code	Same adversarial reviewer + <code>gsd-code-fixer</code> in a review → fix → re-review loop (max 3 iterations)	same	<code>REVIEW.md</code> + atomic fix commits
<code>/gsd-audit-fix --severity high --max 5</code>	Cross-phase audit findings	Wraps an audit source (default <code>audit-uat</code>) with <code>classify</code> → <code>fix</code> → <code>commit</code>	pre-milestone-close	atomic fix commits
<code>/codex challenge (gstack, optional)</code>	Shipped code	Different model family adversarially trying to break your code	after <code>/gsd-code-review --fix --auto</code> , before <code>verify-work</code>	inline report

Rules of thumb:

- **End of each phase** is the primary slot — `/gsd-code-review N --fix --auto` between `execute-phase` and `verify-work`. Running it only at milestone close means a huge diff and the fix commits would land out-of-phase, defeating GSD's atomic-commit-per-phase contract.
- **Pre-execute** is for catching *design* problems, not *code* problems — use `/gsd-review` (one-shot) or `/gsd-plan-review-convergence` (loop) when the phase plan is non-trivial or you want a second opinion before spending execute-time tokens.
- **Pre-milestone-close** is for *aggregate* sweeps that no single phase review can catch — `/gsd-audit-uat` to surface outstanding UAT items and `/gsd-audit-fix --severity high` to auto-remediate.
- The `--depth` flag on `code-review` defaults to `standard`; bump to `deep` for cross-file analysis (import graphs, call chains) once per phase or just before milestone close on the highest-risk phases.

Where the adversarial gates slot in (compact view)

```

plan-phase → [pre-execute adversarial – design level]           → execute-phase → [post-execute
adversarial – code level]   → verify-work                      /gsd-code-
      /gsd-review --phase N --all                               /gsd-code-
review N --fix --auto                                           /codex
      /gsd-plan-review-convergence N --all
challenge (optional, external)
      (loop until no HIGH concern)

... per-phase loop continues until all phases done ...

audit-milestone → [cross-phase aggregate sweep] → complete-milestone
      /gsd-audit-uat
      /gsd-audit-fix --severity high
      /gsd-validate-phase N (per flagged phase, if Nyquist gaps)

```

Diagram 3 — Side workflows · remediation, parallel, retroactive

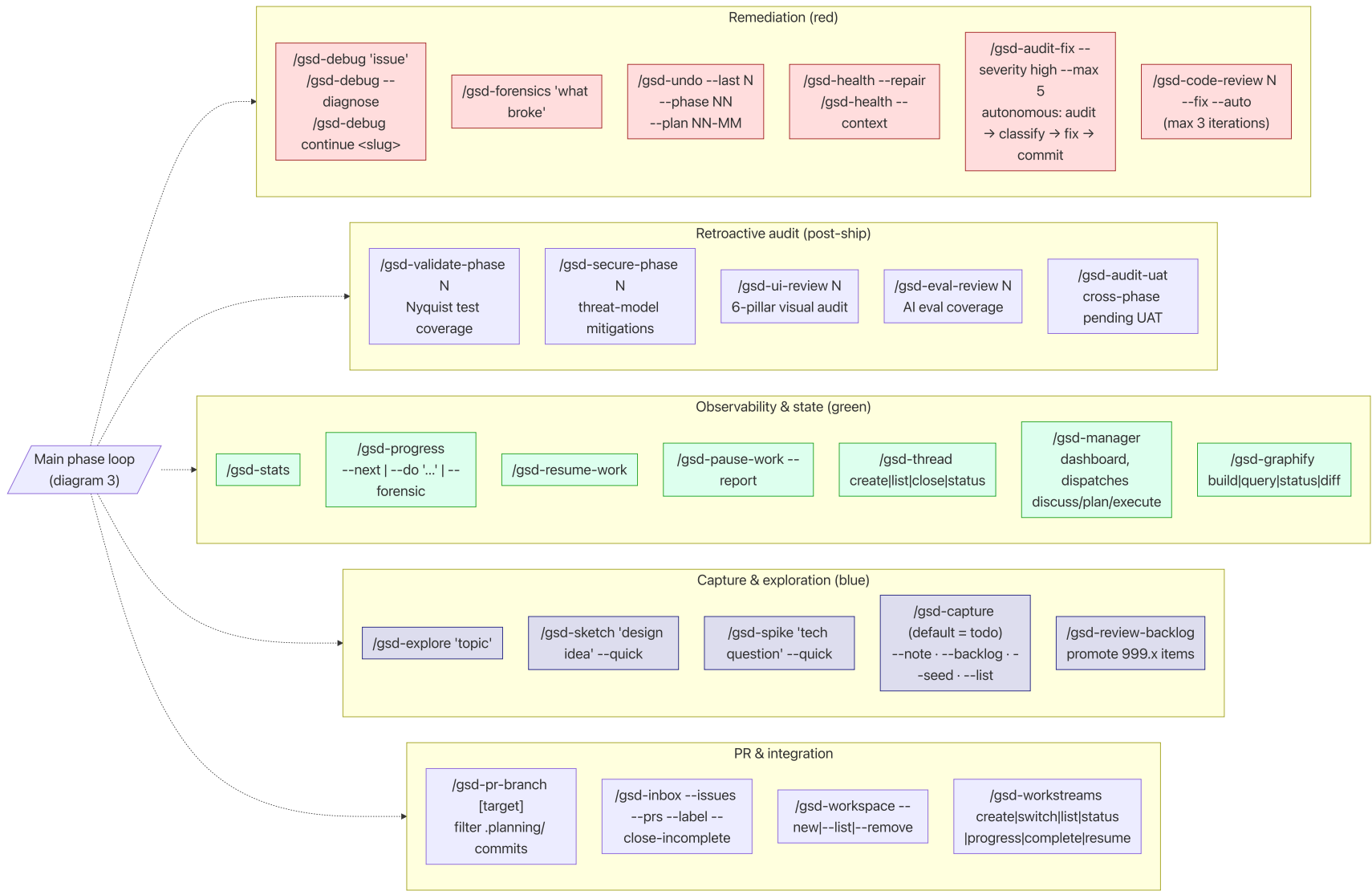


Diagram 4 — Autonomous shortcuts

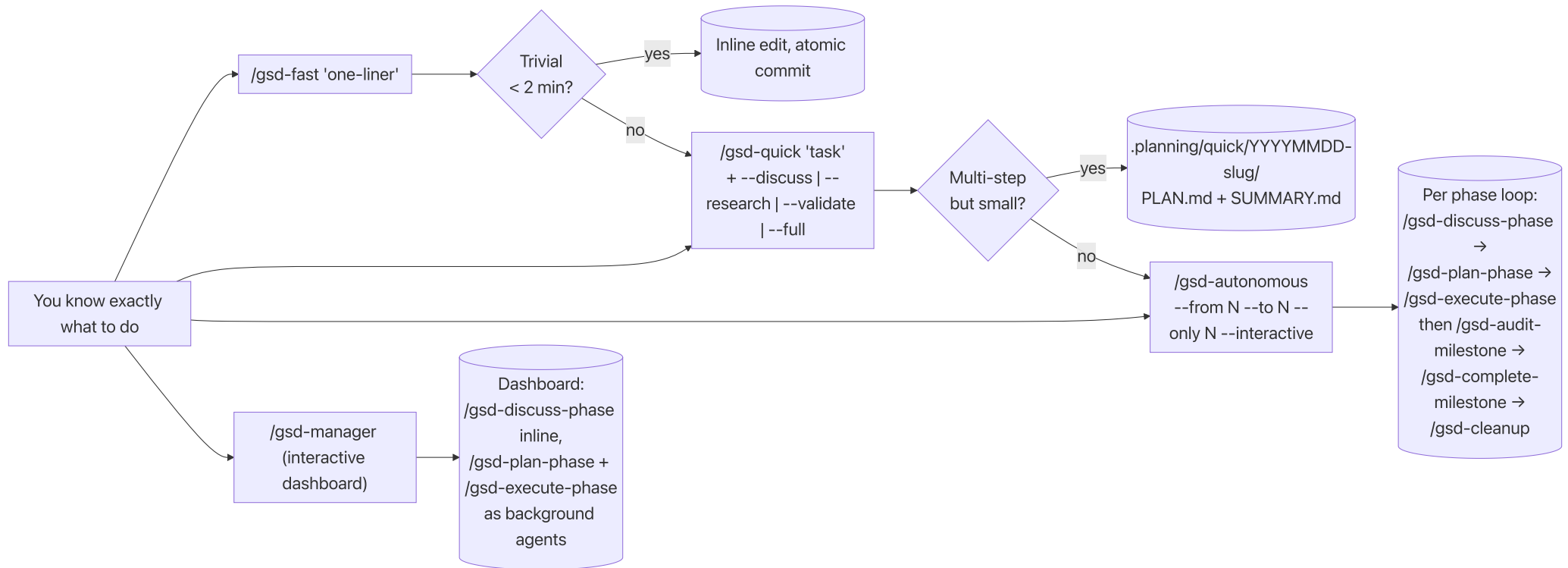
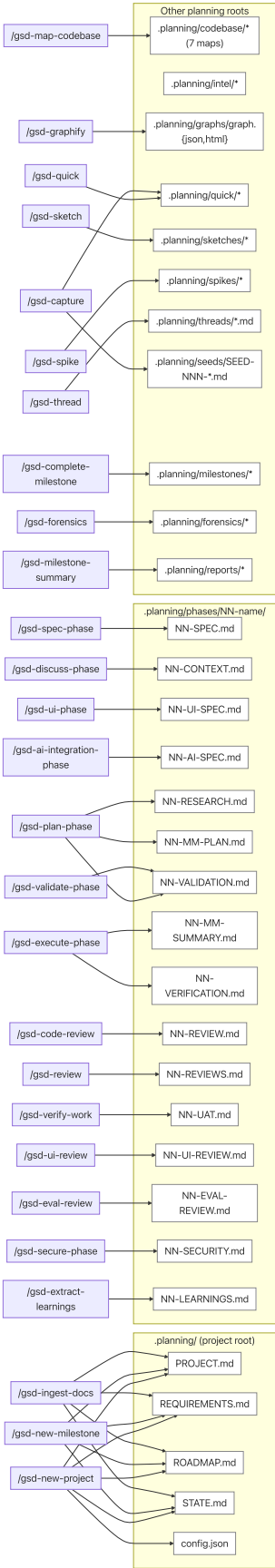


Diagram 5 — Artifact graph · who writes what, who reads what



Namespace routers & flag conventions

Routers exist to keep the eager skill listing small (~120 tokens for the six routers vs ~2,150 for a flat 86-skill listing). You almost never type them — the model uses them to discover the right concrete sub-skill. Calling a router directly just lists its targets and dispatches.

Router	Routes to
/gsd-workflow	discuss · spec · plan · execute · verify · phase · progress · ultraplan · plan-review-convergence
/gsd-project	new-project · new-milestone · complete-milestone · audit-milestone · milestone-summary
/gsd-quality	code-review · code-review --fix · audit-uat · secure-phase · eval-review · ui-review · validate-phase · debug · forensics
/gsd-context	map-codebase (full & --fast & --query) · graphify · docs-update · extract-learnings
/gsd-manage	config · workspace · workstreams · thread · pause-work · resume-work · update · ship · inbox · pr-branch · undo
/gsd-ideate	explore · sketch · spike · spec-phase · capture (todo/note/backlog/seed/list)

Most-used flag conventions

- `--auto` — accept the model's recommended defaults at every gate (new-project, discuss-phase, plan-phase, spec-phase).
- `--text` — plain-text numbered lists instead of TUI menus. Use on `/rc` remote sessions and non-Claude runtimes.
- `--chain` — auto-chain to next workflow step (currently only `discuss-phase` → `plan-phase`).
- `--reviews` — replan applying `REVIEWS.md` feedback from `/gsd-review`. Looped form: `/gsd-plan-review-convergence`.
- `--gaps` / `--gaps-only` — work only on `gap_closure: true` plans. Plug verify-work failures back into `execute-phase --gaps-only`.
- `--ws <name>` — scope to a non-active workstream. `--depth=quick|standard|deep` on code-review. `--profile quality|balanced|budget|inherit` on config.
- Phase arg accepts `12`, `72.1` (inserted urgent), `12a` (letter-suffix). Optional ⇒ auto-detect from `STATE.md`.
- Flag activation rule:** a flag is active *only* when its literal token appears in `$ARGUMENTS`. "Documented" ≠ "default-on".

Detailed reference

Every command, every flag. Sourced verbatim from HELP.md §7, §8, §10.

Standalone commands

These don't depend on the phase loop. They either operate on the whole repo, on `.planning/` state, on git history, or are pure utilities/observability. Most have no required arguments.

Command	Purpose	Required prerequisites
<code>/gsd-help</code>	Show built-in command reference	none
<code>`/gsd-update [--sync\</code>	<code>--reapply]`</code>	Update GSD via npm with changelog
<code>`/gsd-config [--advanced\</code>	<code>--integrations\</code>	<code>--profile <name>]`</code>
<code>/gsd-settings</code>	Original 5-question settings prompt (subset of <code>--config</code>)	none
<code>/gsd-stats</code>	Phases, plans, requirements, git timeline	<code>.planning/</code> exists
<code>/gsd-health [--repair] [--context]</code>	Validate <code>.planning/</code> integrity; <code>--context</code> checks token-window utilisation	<code>.planning/</code> exists
<code>/gsd-forensics ["problem desc"]</code>	Post-mortem investigation of a stuck workflow → <code>.planning/forensics/</code>	git repo
<code>`/gsd-graphify build\</code>	<code>query <term>\</code>	<code>status\</code>
<code>`/gsd-map-codebase [--fast [--focus tech\</code>	<code>arch\</code>	<code>quality\</code>
<code>/gsd-docs-update [--force] [--verify-only]</code>	Generate/refresh up to 9 project doc files	<code>.planning/</code> recommended
<code>`/gsd-ingest-docs [path] [--mode new\</code>	<code>merge] [--manifest <file>] [--resolve auto\</code>	<code>interactive]`</code>
<code>`/gsd-import --from <filepath>\</code>	<code>--from-gsd2 [--path <dir>]`</code>	Import external PLAN.md (with conflict detection) or migrate from GSD-2
<code>/gsd-fast ["task"]</code>	Inline trivial edit, atomic commit, no subagent	git repo
<code>`/gsd-quick [list\</code>	<code>status <slug>\</code>	<code>resume <slug>] [--full\</code>
<code>/gsd-explore ["topic"]</code>	Socratic ideation → routes to note/todo/seed/req/phase	none
<code>`/gsd-sketch ["design idea"\</code>	<code>frontier] [--quick] [--text] [--wrap-up]`</code>	Throwaway HTML mockup variants in <code>.planning/sketches/</code>
<code>`/gsd-spike ["idea"\</code>	<code>frontier] [--quick] [--text] [--wrap-up]`</code>	Time-boxed technical spike in <code>.planning/spikes/</code>
<code>`/gsd-debug ["issue"\</code>	<code>list\</code>	<code>status <slug>\</code>
<code>/gsd-pause-work [--report]</code>	Write <code>.continue-here.md</code> handoff, WIP commit	git repo
<code>/gsd-resume-work</code>	Reconstruct context from STATE.md / checkpoint, route to next action	<code>.planning/</code> exists
<code>`/gsd-thread [list [--open\</code>	<code>--resolved]\</code>	<code>close <slug>\</code>
<code>`/gsd-capture [--note\</code>	<code>--backlog\</code>	<code>--seed\</code>
<code>/gsd-inbox [--issues] [--prs] [--label] [--close-incomplete] [--repo owner/repo]</code>	Triage GitHub issues/PRs against project templates	gh auth
<code>`/gsd-workspace --new\</code>	<code>--list\</code>	<code>--remove [name]`</code>
<code>`/gsd-workstreams list\</code>	<code>create <name>\</code>	<code>switch <name>\</code>

Command	Purpose	Required prerequisites
<code>/gsd-profile-user [--questionnaire] [--refresh]</code>	Generate developer behavioural profile + CLAUDE.md artifacts	session history
<code>/gsd-pr-branch [target=main]</code>	Create clean PR branch with <code>.planning/</code> commits filtered out	git repo
<code>`/gsd-undo --last N\</code>	<code>--phase NN\</code>	<code>--plan NN-MM`</code>
<code>`/gsd-progress [--forensic\</code>	<code>--next\</code>	<code>--do "..."]`</code>
<code>/gsd-cleanup</code>	Archive accumulated phase dirs from completed milestones	<code>.planning/</code> exists
<code>/gsd-review-backlog</code>	Review 999.x backlog and promote/keep/remove items	<code>.planning/</code> exists
<code>/gsd-audit-uat</code>	Cross-phase scan for pending/skipped/blocked UAT items	<code>.planning/phases/</code> exists
<code>`/gsd-audit-fix --source <audit-uat> [--severity high\</code>	<code>medium\</code>	<code>all] [--max N] [--dry-run]`</code>
<code>/gsd-milestone-summary [version]</code>	Human-friendly milestone overview for onboarding	completed milestone
<code>/gsd-manager [--analyze-deps]</code>	Interactive dashboard, parallel <code>discuss/plan/execute</code> dispatch	<code>.planning/</code> exists
<code>/gsd-autonomous [--from N] [--to N] [--only N] [--interactive]</code>	Run all remaining phases unattended	<code>.planning/ROADMAP.md</code> exists

--

Full command reference

Format: `command argument-hint · goal · key flags/subcommands · output`.

Project & milestone lifecycle

Command	Args	Goal	Notable flags / subcommands	Output / artifacts
<code>/gsd-new-project</code>	<code>[--auto]</code>	Greenfield init: questions → research → REQUIREMENTS → ROADMAP	<code>--auto</code> runs research/req/roadmap without interaction (expects PRD via <code>@file</code>)	<code>.planning/PROJECT.md</code> , <code>REQUIREMENTS.md</code> , <code>ROADMAP.md</code> , <code>STATE.md</code> , <code>config.json</code> , <code>research/</code>
<code>/gsd-new-milestone</code>	<code>[milestone name]</code>	Brownfield equivalent: open next milestone	name optional → prompts if absent	Updates <code>PROJECT.md</code> , fresh <code>REQUIREMENTS.md</code> / <code>ROADMAP.md</code> / <code>STATE.md</code> , optional <code>research/</code>
<code>/gsd-audit-milestone</code>	<code>[version]</code>	Verify milestone met definition of done; aggregate VERIFICATION.md + integration check	defaults to current milestone	<code>.planning/v{version}-MILESTONE-AUDIT.md</code>
<code>/gsd-complete-milestone</code>	<code><version></code>	Archive milestone (ROADMAP + REQUIREMENTS) + tag release	recommends <code>/gsd-audit-milestone</code> first	<code>.planning/milestones/v{version}*.md</code> , git tag <code>v{version}</code>
<code>/gsd-milestone-summary</code>	<code>[version]</code>	Onboarding-grade summary from completed milestone artifacts	optional Q&A loop after writing	<code>.planning/reports/MILESTONE_SUMMARY-v{version}.md</code>
<code>/gsd-phase</code>	<code>`[--insert\</code>	<code>--remove\</code>	<code>--edit] <phase-name-or-number>`</code>	CRUD operations on <code>ROADMAP.md</code> phases
<code>/gsd-review-backlog</code>	none	Triage 999.x backlog: promote → active, keep, or remove	interactive	Updates <code>ROADMAP.md</code> , may delete <code>.planning/phases/999.*</code>
<code>/gsd-cleanup</code>	none	Archive completed-milestone phase dirs into <code>.planning/milestones/v{X.Y}-phases/</code>	dry-run first, then confirm	Moves dirs under <code>.planning/milestones/</code>

Pre-planning (per phase, optional)

Command	Args	Goal	Notable flags	Output
<code>/gsd-spec-phase</code>	<code><phase> [--auto] [--text]</code>	Lock WHAT a phase delivers with ambiguity scoring (Socratic; gates ambiguity ≤ 0.20)	<code>--auto</code> selects defaults; <code>--text</code> plain-text menus	<code>{NN}-SPEC.md</code>
<code>/gsd-discuss-phase</code>	<code><phase> [--all] [--auto] [--chain] [--batch] [--analyze] [--text] [--power] [--assumptions]</code>	Lock HOW (gray-area decisions). Required before <code>plan-phase</code>	<code>--all</code> discuss every gray area; <code>--auto</code> defaults; <code>--chain</code> auto-chain to plan; <code>--batch</code> collect-then-decide; <code>--analyze</code> pre-scan; <code>--power</code> deeper Q&A; <code>--assumptions</code> invert: codebase-first assumptions	<code>{NN}-CONTEXT.md</code>

Command	Args	Goal	Notable flags	Output
<code>/gsd-ui-phase</code>	<code>[phase]</code>	UI design contract (spacing, color, typography, copy, registry-safety) before plan	shadcn init gate for React/Next/Vite	<code>{NN}-UI-SPEC.md</code>
<code>/gsd-ai-integration-phase</code>	<code>[phase]</code>	AI design contract: framework choice + docs + domain research + eval strategy	orchestrates 4 sub-agents	<code>{NN}-AI-SPEC.md</code>
<code>/gsd-mvp-phase</code>	<code><phase></code>	Vertical-slice MVP planning with SPIDR split check (writes Mode: mvp)	delegates to <code>/gsd-plan-phase</code>	Updates <code>ROADMAP.md</code> , may emit <code>SKELETON.md</code> on Phase 1

Planning, review, execution

Command	Args	Goal	Notable flags	Output
<code>/gsd-plan-phase</code>	<code>[phase] [--auto] [--research] [--skip-research] [--research-phase <N>] [--view] [--gaps] [--skip-verify] [--prd <file>] [--reviews] [--text] [--tdd] [--mvp]</code>	Research → plan → verify loop (max ~3 iterations)	<code>--research</code> force re-research; <code>--skip-research</code> straight to plan; <code>--research-phase N</code> research-only stop; <code>--view</code> print existing <code>RESEARCH.md</code> ; <code>--gaps</code> gap-closure mode (reads <code>VERIFICATION.md</code>); <code>--skip-verify</code> no plan-checker; <code>--prd <file></code> use PRD instead of <code>CONTEXT.md</code> ; <code>--reviews</code> apply <code>REVIEWS.md</code> feedback; <code>--text</code> non-TUI menus; <code>--tdd</code> TDD mode; <code>--mvp</code> vertical slice	<code>{NN}-RESEARCH.md</code> , <code>{NN}-VALIDATION.md</code> , <code>{NN}-MM-PLAN.md</code> files
<code>/gsd-review</code>	<code>--phase N [--gemini] [--claude] [--codex] [--opencode] [--qwen] [--cursor] [--all]</code>	Cross-AI peer review (external CLIs read the plans)	<code>--all</code> runs every detected CLI	<code>{NN}-REVIEWS.md</code>
<code>/gsd-plan-review-convergence</code>	<code><phase> [--codex] [--gemini] [--claude] [--opencode] [--ollama] [--lm-studio] [--llama-cpp] [--text] [--ws <name>] [--all] [--max-cycles N]</code>	Replan → review → replan loop until no HIGH concerns (or max cycles)	wraps <code>plan-phase --reviews + review</code>	Updated <code>PLAN.md</code> files, <code>REVIEWS.md</code>
<code>/gsd-ultraplan-phase</code>	<code>[phase]</code>	BETA — offload planning to Claude Code <code>ultraplan</code> cloud	requires CC v2.1.91+, claude.ai account, GitHub repo	Cloud plan, imported via <code>/gsd-import --from</code>
<code>/gsd-import</code>	<code>`--from <file>\`</code>	<code>--from-gsd2 [--path <dir>]`</code>	Import external plan with conflict detection vs <code>PROJECT.md</code> , or reverse-migrate GSD-2 → v1	runs <code>gsd-plan-checker</code> on imported plan
<code>/gsd-execute-phase</code>	<code><phase> [--wave N] [--gaps-only] [--interactive] [--tdd]</code>	Wave-based parallel execution; fresh 200K context per plan	<code>--wave N</code> only one wave; <code>--gaps-only</code> only <code>gap_closure: true</code> plans; <code>--interactive</code> sequential pair-programming style	<code>{NN}-MM-SUMMARY.md</code> per plan, <code>{NN}-VERIFICATION.md</code> aggregate

Command	Args	Goal	Notable flags	Output
<code>/gsd-add-tests</code>	<code><phase> [additional instructions]</code>	Supplement only — phase already has tests from <code>execute-phase</code> (per-task <code><verify></code> blocks); use this for extra fuzz / edge / regression cases. RED-GREEN classification.	free-text extras (e.g. "focus on pricing edge cases")	Test files committed as <code>test(phase-{N}): ...</code>
<code>/gsd-verify-work</code>	<code>[phase]</code>	Conversational UAT, one test at a time; diagnoses failures → fix plans for <code>execute-phase --gaps-only</code>	optional phase number; prompts if missing	<code>{NN}-UAT.md</code> (+ fix plans in phase dir)
<code>/gsd-ship</code>	<code>`[phase]</code>	milestone, e.g. '4' or 'v1.0']`	Push branch, create PR (auto body), optionally trigger review, track merge	works on phase or milestone scope

Code-quality & audits

Command	Args	Goal	Notable flags	Output
<code>/gsd-code-review</code>	<code>`<phase> [--depth=quick]</code>	standard\	<code>deep</code> [--files f1,f2,...] [--fix [--all] [--auto]]`	Bug/security/quality review of phase-changed files
<code>/gsd-audit-fix</code>	<code>`[--source <audit-uat>] [--severity high]</code>	medium\	<code>all</code> [--max N] [--dry-run]`	Autonomous audit → classify auto-fixable vs manual → fix + test + commit
<code>/gsd-audit-uat</code>	none	Cross-phase scan of pending UAT items, prioritized human test plan	none	Inline report (no file)
<code>/gsd-secure-phase</code>	<code>[phase]</code>	Threat-mitigation audit. Pre-ship gate when <code>workflow.security_enforcement: true</code> (default) — <code>verify-work</code> won't auto-complete the phase without <code>SECURITY.md</code> & <code>threats_open: 0</code> . Retroactive/optional when the toggle is off.	three states: (A) <code>SECURITY.md</code> exists → audit; (B) <code>PLAN</code> threat model exists → run from artifacts; (C) phase not executed → exit	<code>{NN}-SECURITY.md</code> (with <code>threats_open</code> count)
<code>/gsd-validate-phase</code>	<code>[phase]</code>	Nyquist coverage audit; generates missing tests (auditor never edits impl). Pre-ship coverage is enforced by plan-checker's 8th dim when <code>workflow.nyquist_validation: true</code> (default), so this command is mostly retroactive — used for phases shipped before Nyquist was enabled, or brownfield code.	escalates impl bugs to user	Updates <code>{NN}-VALIDATION.md</code> + new test files
<code>/gsd-eval-review</code>	<code>[phase]</code>	Retroactive AI-eval coverage audit vs <code>AI-SPEC.md</code>	for phases built with <code>ai-integration-phase</code>	<code>{NN}-EVAL-REVIEW.md</code>
<code>/gsd-ui-review</code>	<code>[phase]</code>	Retroactive 6-pillar visual audit (works on any project)	Playwright screenshots in <code>.planning/ui-reviews/</code>	<code>{NN}-UI-REVIEW.md</code>
<code>/gsd-extract-learnings</code>	<code><phase></code>	Decisions / lessons / patterns / surprises from finished phase artifacts	none	<code>{NN}-LEARNINGS.md</code>

Debug, observability, state

Command	Args	Goal	Notable flags / subcommands	Output
/gsd-debug	`[list\`	status <slug>\`	continue <slug>\`	--diagnose] [issue]`
/gsd-forensics	["problem"]	Post-mortem of a stuck workflow (git log + .planning/ + fs state)	optional GitHub issue creation	.planning/forensics/{ts}-{slug}.md
/gsd-health	[--repair] [-context]	Validate .planning/ integrity; --context reports current session token utilisation (60/70 % thresholds)	--repair auto-fixes structural issues	inline report; may write fixes
/gsd-stats	none	Phases, plans, requirements, git metrics, timeline	none	inline report
/gsd-progress	`[--forensic\`	--next\`	--do "task"`	Status + smart-routes to next action; --do matches freeform intent to a command
/gsd-manager	[--analyze-deps]	Interactive multi-phase dashboard; background plan/execute, inline discuss	--analyze-deps runs dependency-analysis workflow once	none directly; dispatches existing commands
/gsd-resume-work	none	Reconstruct context (STATE.md, checkpoints, incomplete PLANS) and route	none	inline; may dispatch next command
/gsd-pause-work	[--report]	Write .continue-here.md, WIP commit; --report includes session summary	none	.continue-here.md + WIP commit
/gsd-thread	`[list [--open\`	--resolved]\`	close <slug>\`	status <slug>\`
/gsd-capture	`[--note\`	--backlog\`	--seed\`	--list] [text]`
/gsd-undo	`--last N\`	--phase NN\`	--plan NN-MM`	Safe revert via phase manifest with dependency check + confirm gate

Codebase intelligence & docs

Command	Args	Goal	Notable flags	Output
/gsd-map-codebase	`[--fast [--focus tech\`	arch\`	quality\`	concerns\`
/gsd-graphify	`build\`	query <term>\`	status\`	diff`
/gsd-docs-update	[--force] [--verify-only]	Generate/refresh up to 9 project doc files via gsd-doc-writer per doc	--force overwrites everything; --verify-only no writes, just check	up to 9 files (README, ARCHITECTURE, CONTRIBUTING, ...)
/gsd-ingest-docs	`[path] [--mode new\`	merge] [--manifest <file>] [-resolve auto\`	interactive]`	Build/merge .planning/ from ADRs/PRDs/SPECs/RFCs (precedence ADR > SPEC > PRD > DOC)

Workspaces, threads, capture, exploration

Command	Args	Goal	Notable flags / subcommands	Output
/gsd-workspace	`[--new\	--list\	--remove] [name]`	Manage isolated workspace dirs (separate worktrees/clones)
/gsd-workstreams	`list\	create <name>\	status <name>\	switch <name>\
/gsd-explore	["topic"]	Open-ended Socratic ideation → routes to note/todo/seed/req/phase	optional topic	varies by route
/gsd-sketch	`["idea"\	frontier] [--quick] [--text] [--wrap-up]`	2-3 HTML mockup variants in .planning/sketches/ (no build)	--quick skip mood intake; --wrap-up package as project skill
/gsd-spike	`["idea"\	frontier] [--quick] [--text] [--wrap-up]`	Experiential validation experiments (GivenWhenThen + VALIDATED verdict)	same as sketch

GitHub, PR, config, lifecycle utilities

Command	Args	Goal	Notable flags	Output
/gsd-pr-branch	[target=main]	Clean PR branch — strip .planning/ commits so reviewers see only code	optional target branch	new branch, push instructions
/gsd-inbox	[--issues] [--prs] [--label] [--close-incomplete] [--repo owner/repo]	Triage GitHub issues/PRs vs project templates	--label auto-apply labels; --close-incomplete close non-compliant	inline report, optional comments/labels
/gsd-ship	see above			
/gsd-config	`[--advanced\	--integrations\	--profile <name>]`	Consolidated settings interface
/gsd-settings	none	Legacy 5-question config prompt (subset of --config)	none	Writes .planning/config.json
/gsd-update	`[--sync\	--reapply]`	npm update with changelog display	--sync re-sync without bumping; --reapply reinstall same version
/gsd-profile-user	[--questionnaire] [--refresh]	Build developer profile from session analysis (or questionnaire fallback)	--refresh rebuild + diff	USER-PROFILE.md, /gsd-dev-preferences, CLAUDE.md section
/gsd-help	none	Display the built-in command reference	none	inline
/gsd-fast	["task"]	Trivial inline edit, no PLAN, no subagent	freeform task	atomic commit
/gsd-quick	`[list\	status <slug>\	resume <slug>\	--full] [--validate] [--discuss] [--research] [task]`
/gsd-autonomous	[--from N] [--to N] [--only N] [--interactive]	Run remaining phases unattended; loops discuss→plan→execute per phase; then audit + complete + cleanup	--interactive runs discuss inline, plan/execute as background agents	All phase artifacts, milestone archive

Argument & flag conventions

A few consistent patterns appear across the surface — knowing them lets you read any new command's frontmatter at a glance.

- **Phase argument** — most phase-scoped commands accept an integer (1, 12), a decimal (72.1 for inserted urgent work), or a letter-suffix (12a). When the phase is **optional**, the command auto-detects the next unplanned/unverified phase from `STATE.md` + `ROADMAP.md`.
- **Flag activation rule** — every command states it explicitly: *> A flag is active only when its literal token appears in \$ARGUMENTS.*

Do not assume "documented" means "default-on".

- **--auto** — accept the model's recommended defaults at every gate. Available on `new-project`, `discuss-phase`, `plan-phase`, `spec-phase`.
- **--text** — use plain-text numbered lists instead of TUI menus (`AskUserQuestion`). Required for `/rc` remote sessions and non-Claude runtimes that lack interactive widgets.
- **--chain** — auto-chain to the next workflow step (currently only on `discuss-phase` → `plan-phase`). `/gsd-autonomous` is the milestone-wide generalisation.
- **--reviews** (planning) — replan applying feedback from `REVIEWS.md` produced by `/gsd-review`. `plan-review-convergence` is the looped form.
- **--gaps / --gaps-only** — work only on `gap_closure: true` plans. Plug `verify-work` failures back into `execute-phase --gaps-only`.
- **--ws <name>** — scope a command to a non-active workstream. Honoured by workstream-aware commands (`autonomous`, `plan-review-convergence`, `resume-work`, ...).
- **--depth=quick|standard|deep** (code-review) — overrides the `workflow.code_review_depth` config; only takes effect for that invocation.
- **--profile quality|balanced|budget|inherit** (config) — switches the per-agent model assignment table. `inherit` defers to the runtime default (useful for non-Claude CLIs).
- **Subcommand-style** — `debug`, `quick`, `thread`, `workstreams`, `graphify`, `map-codebase`, `workspace`, `capture` accept positional subcommands (`list`, `status <slug>`, `continue <slug>`, `resume <slug>`, etc.) **before** any **--** flag. Reading the `argument-hint` from `commands/gsd/<cmd>.md` is always the authoritative source.

End of `HELP.md`. For canonical docs see `README.md`, `docs/USER-GUIDE.md`, `docs/COMMANDS.md`, `docs/ARCHITECTURE.md`, `docs/FEATURES.md`, `docs/INVENTORY.md`.